作品紹介

作品名:BarrierCounter

ジャンル:3Dアクション

開発環境:DXライブラリ、Visual Studio 2022

言語:C++

制作期間:1か月

制作人数:1人

担当箇所:プログラム全般



動作環境:Windows(XInputコントローラー対応)

GitHub:https://github.com/KaminagaAyumu/BarrierCounter.git

動画:https://youtu.be/N-rrD2n-XXA

ゲーム概要

・バリアを使って敵を倒していく3Dアクションゲーム
・敵を全員倒せばステージクリア
・クリアタイムによってスコアボーナスが増える





操作方法

左スティック:移動　右スティック:カメラ操作 Aボタン:バリア Xボタン:マップを見る


こだわった点

チュートリアルムービーの実装

ゲーム開始前に簡単な説明ムービーを入れ、ユーザーがどのようなゲームなのかを一目で理解できるようにしました。

動画とテキストでゲームの流れを説明



説明が不要なユーザーのためにスキップ機能も実装



アニメーション

アニメーションブレンドを行い、急にアニメーションが切り替わらないようにこだわりました。

1フレーム単位での動き
ゆっくり右手が前に出てきている








技術アピール

挙動の一括化

ゲーム中のオブジェクトをすべてCollidableクラスを基底にすることにより、位置の更新や当たり判定を一括化させることができました。

Collidableクラスは当たり判定や移動の更新に必要なデータを持っている

```
protected:
    // 物理挙動関連データ
    Rigidbody m_rigidbody;
    // 当たり判定関連データ
    std::shared_ptr<ColliderBase> m_pColliderData;
    // 未来の座標
    Vector3 m_nextPos;

};
```

管理用クラスのPhisicsに登録、解除することで一括管理が行える

```
void Physics::Entry(std::shared_ptr<Collidable> collidable)
{
    // 既に登録されているかどうかを確認
    bool isFound = (std::find(m_pCollidables.begin(), m_pCollidables.end(), collidable) != m_pCollidables.end());

    if (isFound)
    {
        // すでに登録されていた場合アサート
        assert(false && "当たり判定がすでに登録済みです");
    }
    else
    {
        // 登録する
        m_pCollidables.emplace_back(collidable);
    }
}

void Physics::Exit(std::shared_ptr<Collidable> collidable)
{
    // 登録されているCollidableを確認
    auto count = std::erase_if(m_pCollidables, [collidable](std::shared_ptr<Collidable> target)
        {
            // 対象のCollidableと一致したものを消去
            return target == collidable;
        });
}
```

リソース管理

エフェクトのリソース管理の方法を参照カウンタ方式にして、必要のないメモリ確保を行わないようにしました。

```cpp
void EffectManager::LoadEffect(const std::wstring& path)
{
    // エフェクトのパスがすでにロード済みの場合
    if (m_effectResources.contains(path))
    {
        // リソースの参照カウンタを増加
        m_effectResources[path].refCounter++;
    }
    else
    {
        // リソースの参照カウンタを1にする
        m_effectResources[path].refCounter = kResourceExist;
        // パスに対応したエフェクトハンドルを取得
        m_effectResources[path].handle = LoadEffekseerEffect((kCommonPathName + path).c_str());
        // パスが見つからなかったらアサート
        assert(m_effectResources[path].handle >= 0 && "エフェクトのパスが存在しません");
    }
}

int EffectManager::DeleteEffect(const std::wstring& path)
{
    // 指定されたパスの要素を返す
    auto it = m_effectResources.find(path);
    assert(it != m_effectResources.end() && "エフェクト削除時にパスからハンドルの取得に失敗しました");

    // 参照カウンタを減らして、最後の参照だったら
    if (--it->second.refCounter <= 0)
    {
        // リソースのハンドルを開放
        DeleteEffekseerEffect(it->second.handle);
        // コンテナからも削除
        m_effectResources.erase(it);
        return 0; // 削除成功
    }
    else
    {
        return kResourceExist; // 削除しないとする
    }
}
```